

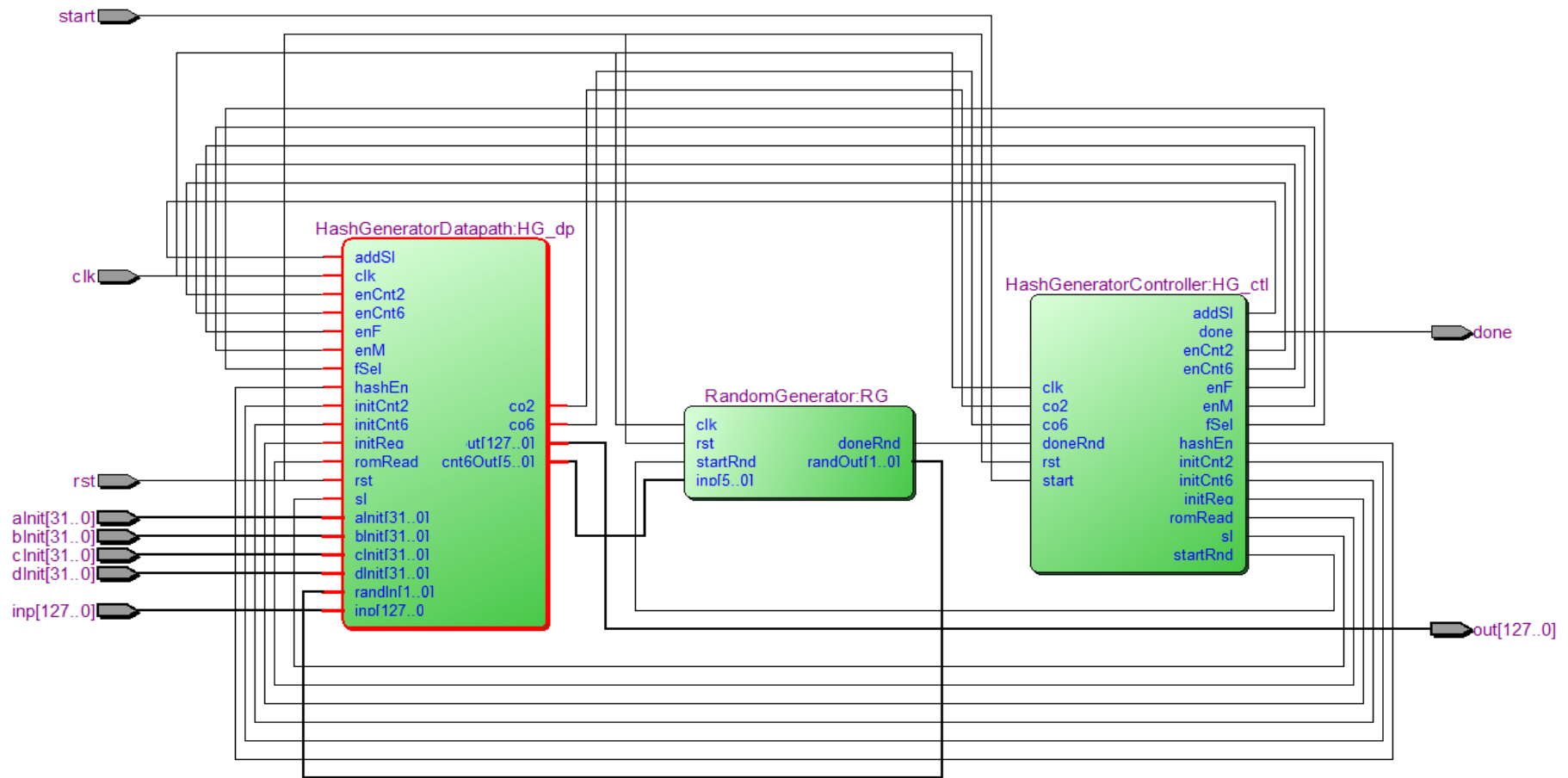


MD5 HASH GENERATOR - DIGITAL DESIGN IMPLEMENTATION REPORT

EXECUTIVE SUMMARY

THIS REPORT DOCUMENTS THE IMPLEMENTATION OF A COMPLETE MD5 HASH GENERATOR SYSTEM DESIGNED USING VERILOG HDL. THE SYSTEM FEATURES A MODULAR ARCHITECTURE WITH SEPARATE CONTROLLER AND DATAPATH COMPONENTS, IMPLEMENTING THE MD5 CRYPTOGRAPHIC HASH ALGORITHM WITH 128-BIT OUTPUT. THE DESIGN HAS BEEN VERIFIED THROUGH SIMULATION TESTING AGAINST KNOWN TEST VECTORS.

1. SYSTEM ARCHITECTURE



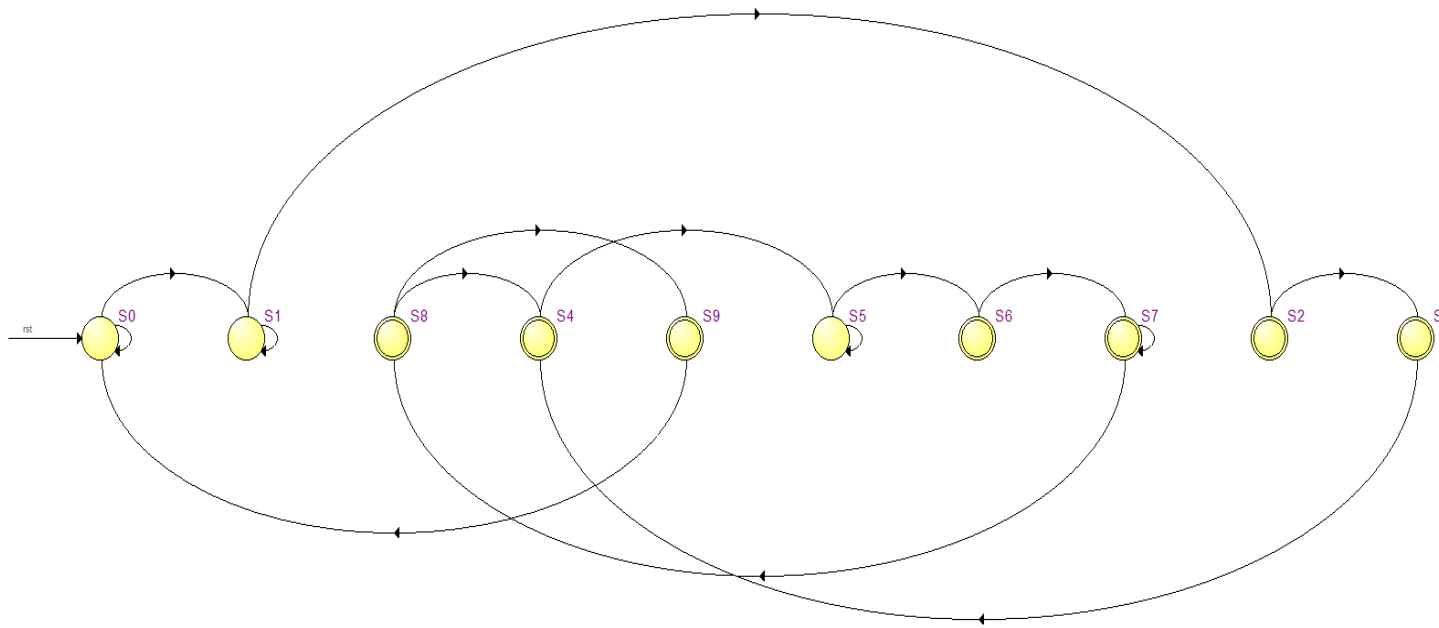
1.1 TOP-LEVEL MODULE: HASHGENERATOR

- **Parameters:** Configurable input size (default: 128-bit)
- **Inputs:** Clock, reset, start signal, 128-bit input data, initial hash values (A, B, C, D)
- **Outputs:** 128-bit hash output, completion signal
- **Submodules:**
 - HashGeneratorController (Finite State Machine)
 - HashGeneratorDatapath (Computational pipeline)
 - RandomGenerator (Random number generation for message scheduling)

The background is a blue gradient with decorative white circuit-like lines in the corners. The lines consist of straight segments and small circles, resembling a printed circuit board layout. They are located in the top-left, top-right, bottom-left, and bottom-right corners.

MODULE SPECIFICATIONS

2.1 HASHGENERATORCONTROLLER



	Source State	Destination State	Condition
1	S0	S1	(start)
2	S0	S0	(!start)
3	S1	S1	(start)
4	S1	S2	(!start)
5	S2	S3	
6	S3	S4	
7	S4	S5	
8	S5	S5	(!doneR...
9	S5	S6	(doneRnd)
10	S6	S7	
11	S7	S7	(!co2)
12	S7	S8	(co2)
13	S8	S4	(!co6)
14	S8	S9	(co6)
15	S9	S0	

Function: Implements the main control FSM for MD5 computation

State Transitions:

- **S0:** Idle state, waits for start signal
- **S1:** Start acknowledgment
- **S2:** Enable message registers, initialize counter
- **S3:** Initialize hash registers
- **S4:** Start random number generation
- **S5:** Wait for random generation completion
- **S6:** Initialize F function computation
- **S7:** Process F function with counter
- **S8:** Update hash values, check loop completion
- **S9:** Output final hash, return to idle

Control Signals:

enM: Enable message registers

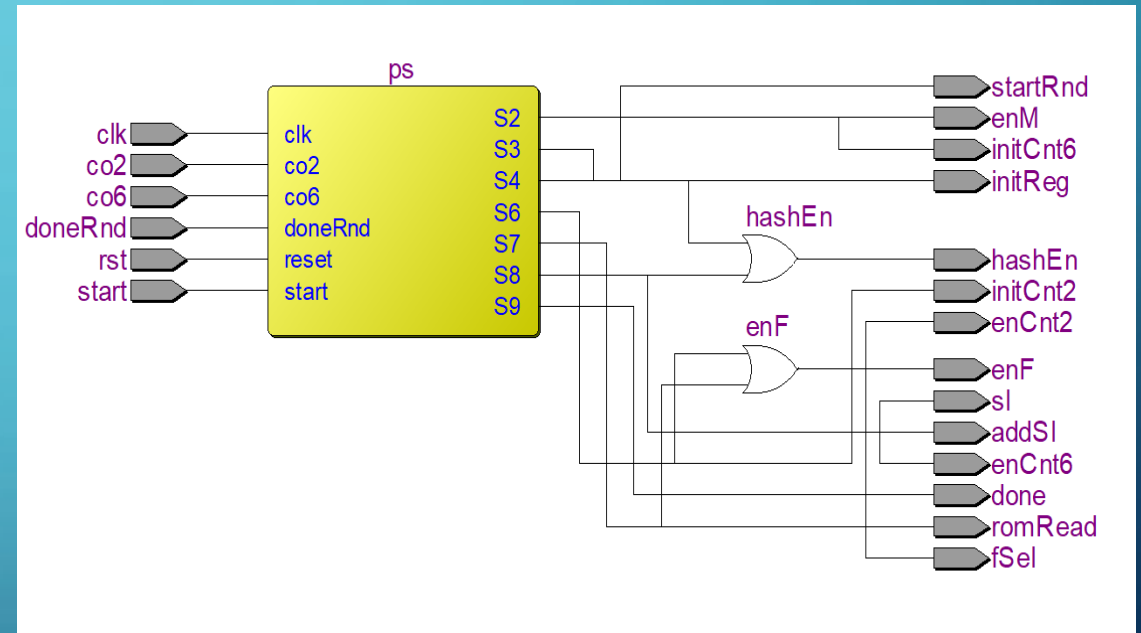
initCnt6: Initialize main loop counter (64 iterations)

hashEn: Enable hash register updates

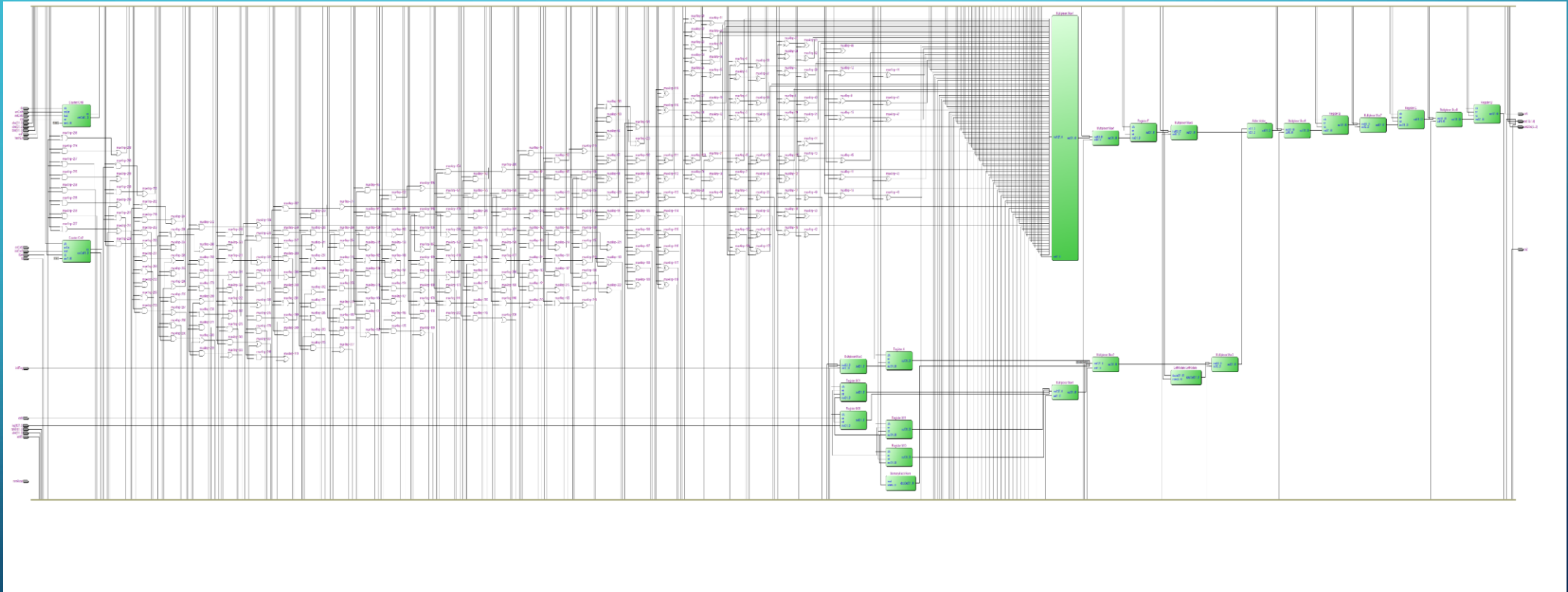
startRnd: Start random number generation

romRead: Enable ROM read operations

And ...



2.2 HASHGENERATORDATAPATH



Function: Implements the computational pipeline for MD5 algorithm

Key Components:

Message Registers (M00-M11):

Four 32-bit registers storing input message blocks

Enabled by enM control signal

Hash Registers (A, B, C, D):

Four 32-bit registers storing intermediate hash values

Support initialization and iterative updates

F Function Logic:

Four logical functions selected based on round number:

Round 1-16: $F(B,C,D) = (B \& C) \mid (\sim B \& D)$

Round 17-32: $G(B,C,D) = (D \& B) \mid (\sim D \& C)$

Round 33-48: $H(B,C,D) = B \wedge C \wedge D$

Round 49-64: $I(B,C,D) = C \wedge (B \mid \sim D)$

Rotation Unit:

Implements variable left rotation based on MD5 specification

Pre-programmed rotation amounts for each round

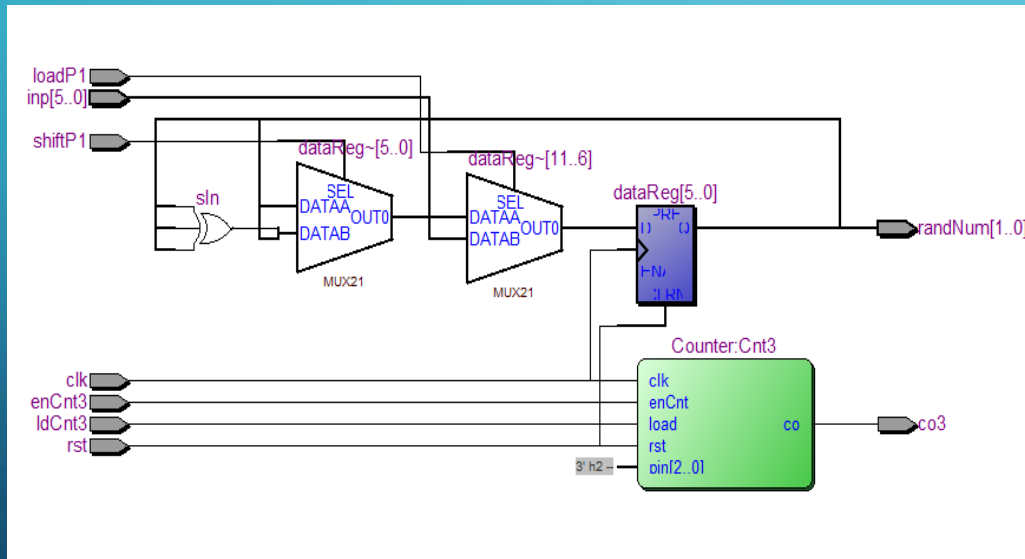
Constants ROM:

Stores 64 32-bit constants for MD5 computation

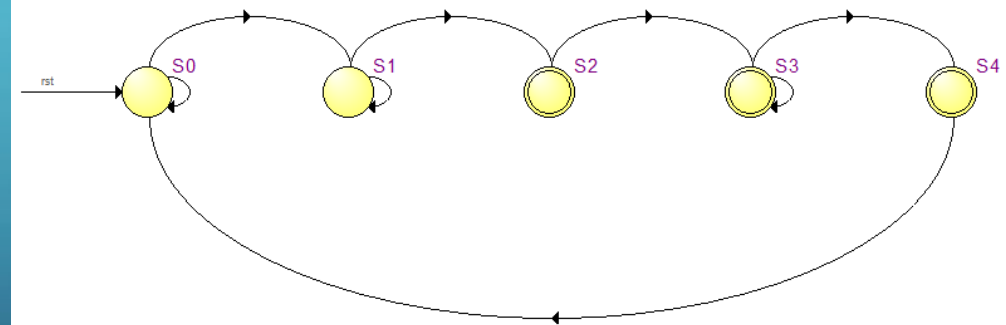
Addressable by main loop counter

2.3 RANDOMGENERATOR

- DATAPATH



- CONTROLLER



	Source State	Destination State	Condition
1	S0	S0	(!startRnd)
2	S0	S1	(startRnd)
3	S1	S2	(!startRnd)
4	S1	S1	(startRnd)
5	S2	S3	
6	S3	S3	(!co3)
7	S3	S4	(co3)
8	S4	S0	

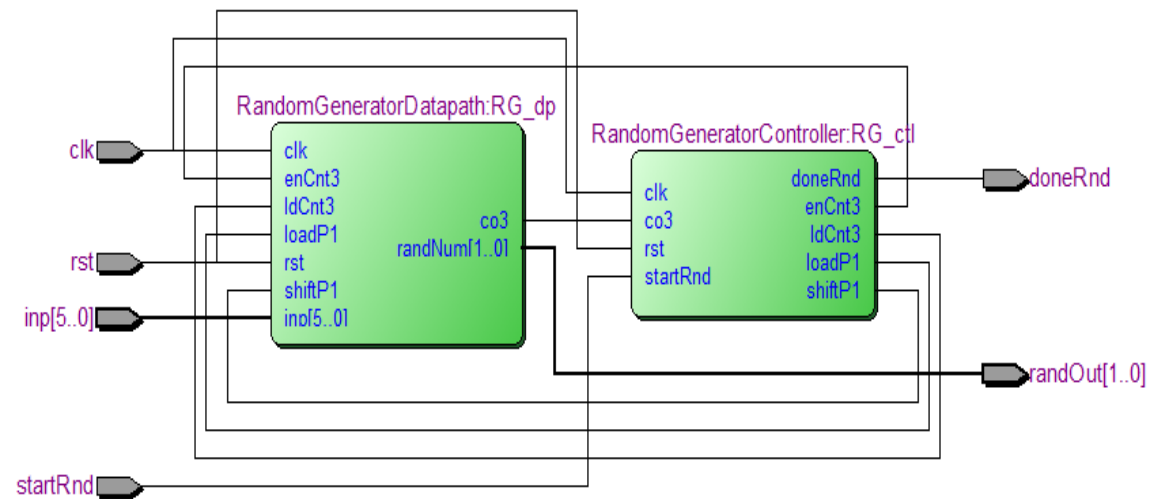
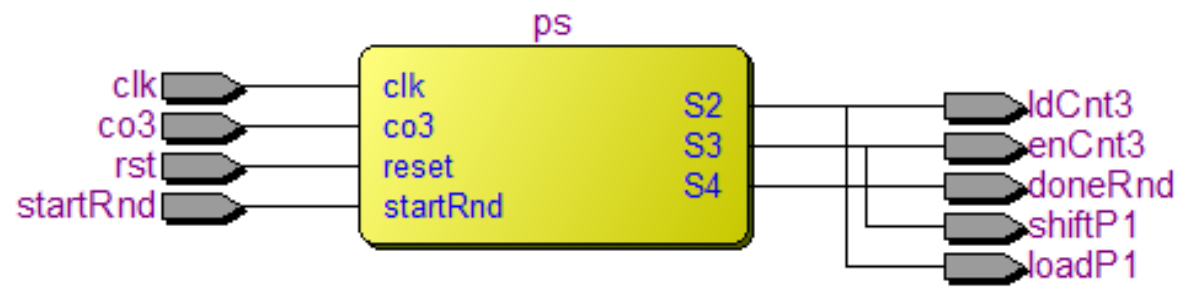
Function: Generates pseudo-random indices for message scheduling

Components:

Controller: 5-state FSM for random generation sequence

Datapath: Linear feedback shift register (LFSR) based implementation

Output: 2-bit random number for message word selection



3. ALGORITHM IMPLEMENTATION

Initialize:

```
a0 := 0x67452301
b0 := 0xefcdab89
c0 := 0x98badcfe
d0 := 0x10325476
```

Break 128-bit message into four 32-bit words $M[0..3]$

```
A := a0
B := b0
C := c0
D := d0
```

for i from 0 to 63 do

 if $0 \leq i \leq 15$ then

$F := (B \text{ and } C) \text{ or } ((\text{not } B) \text{ and } D)$

 else if $16 \leq i \leq 31$ then

$F := (D \text{ and } B) \text{ or } ((\text{not } D) \text{ and } C)$

 else if $32 \leq i \leq 47$ then

$F := B \text{ xor } C \text{ xor } D$

 else if $48 \leq i \leq 63$ then

$F := C \text{ xor } (B \text{ or } (\text{not } D))$

$x := i$

 repeat 6 times:

$fb := (\text{bit5 of } x) \text{ xor } (\text{bit3 of } x) \text{ xor } (\text{bit1 of } x)$

$x := (x \ll 1) \text{ or } fb$

$rnd := (\text{bit5 of } x \text{ shifted left by 1}) \text{ OR } (\text{bit4 of } x)$

$F := F + A + \text{constant}[i] + M[rnd]$

$A := D$

$D := C$

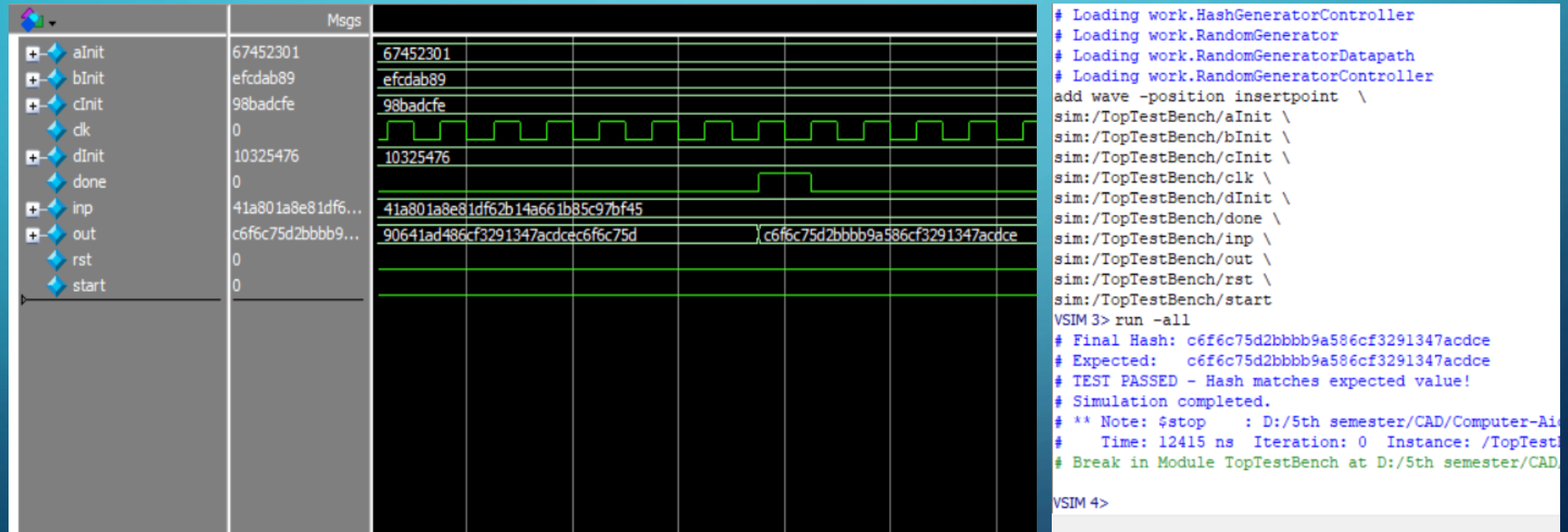
$C := B$

$B := B + \text{leftrotate}(F, \text{step}[i])$

end for

digest := A || B || C || D

PASSES VERIFICATION AGAINST STANDARD TEST VECTORS



CONCLUSION

The implemented MD5 hash generator successfully demonstrates a complete cryptographic hash function in hardware. The modular design approach with separate controller and datapath components provides maintainability and scalability. The system correctly implements the MD5 algorithm specification and passes verification against standard test vectors, confirming functional correctness.

The design represents a balanced approach between performance and resource utilization, making it suitable for integration into larger cryptographic systems or as a standalone hash computation unit.



<https://github.com/MiladMazaheri83>



<https://github.com/Parsa-Saeednia>

Document Version: 1.0

Verification Status: Tested and Verified

Date: 2025/9/10

Designer: Milad Mazaheri , Parsa Saeednia